Vendor–Hiring Manager Contract Management Module

(Production-Grade Multi-Tenant + AI Agent Assignment)

🎯 Objective

Build a **multi-tenant contract hiring module** supporting:

1. **IT Vendor Persona**
2. **Hiring Manager Persona**
3. **AI Recommendation Agent**
4. **Strict RBAC**
5. **Strict latency constraints**
6. **Production-grade architecture**

This is NOT a CRUD assignment.

This must reflect:

- **Clean architecture**
- **Multi-role security**
- **AI agent design (not LLM wrapper)**
- **Observability**
- **Deterministic scoring logic**
- **Performance awareness**
- **Transaction integrity**

👥 Persona 1 – IT Vendor

📝 IT Vendor User Story

I Want To:

• View available contract openings
• View opening details
• Upload multiple candidate profiles (PDF, PPTX)

www.zelosify.com

• Soft delete profiles
• Securely submit profiles

So That:

I can participate in filling contractual openings while ensuring secure, tenant-isolated file sharing.

Authentication & Authorization

Role

Role: IT_VENDOR

Rules

• Can view openings under their tenant
• Can upload profiles
• Can view their own uploads
• Cannot view other vendors' uploads
• Cannot view AI recommendation
• Cannot shortlist/reject

Mandatory Enforcement

• API-level RBAC
• UI-level route guards
• Tenant-based query filtering
• No bypassable endpoints

Failure → Automatic rejection.

Database Schema

Opening Model

```
model Opening {
  id                String   @id @default(uuid())
  tenantId          String
  title             String
  description       String?
  location          String?
  contractType      String?
  hiringManagerId   String
```

www.zelosify.com

```
  experienceMin          Int
  experienceMax          Int?
  postedDate             DateTime @default(now())
  expectedCompletionDate  DateTime?
  actionDate             DateTime?
  status                 OpeningStatus @default(OPEN)

  tenant                 Tenants @relation(fields: [tenantId], references: [tenantId])
  hiringProfiles         hiringProfile[]

  @@index([tenantId])
}
```

Profile Model

```
model hiringProfile {
  id                     Int      @id @default(autoincrement())
  openingId              String
  s3Key                  String   @unique
  uploadedBy             String
  submittedAt            DateTime  @default(now())
  status                 ProfileStatus  @default(SUBMITTED)

  shortlistedBy          String?
  shortlistedAt          DateTime?
  rejectedBy             String?
  rejectedAt             DateTime?

  // AI Agent Fields
  recommended            Boolean?
  recommendationScore    Float?
  recommendationReason   String?
  recommendationLatencyMs  Int?
  recommendationVersion  String?
  recommendationConfidence  Float?
  recommendedAt          DateTime?

  isDeleted              Boolean   @default(false)

  opening                Opening   @relation(fields: [openingId], references: [id])

  @@index([openingId])
  @@index([recommended])
}
```

Enums

```
enum OpeningStatus {
  OPEN
  CLOSED
  ON_HOLD
}

enum ProfileStatus {
```

www.zelosify.com

SUBMITTED
SHORTLISTED
REJECTED
}

Seeding Requirements

• Seed must pre-populate:

- Tenant: "Bruce Wayne Corp"
- At least 12 openings
- Different roles, experience ranges, contract types

• All seed data must belong to the same tenant.

Tenant isolation must remain intact.

IT Vendor Backend APIs

1.Fetch Openings

GET /api/vendor/openings

Pagination required.

Tenant filtering mandatory.

2.Fetch Opening Details

GET /api/vendor/openings/:id

Must include:
• Hiring Manager name
• Experience range
• Profiles count
• Uploaded profiles list

---

www.zelosify.com

3.Presign Upload URLs

POST /api/vendor/openings/:id/profiles/presign

Profiles stored under:

<bucket>/<tenantId>/<openingId>/<timestamp>_<filename>

No frontend → direct S3 access.

4.Submit Profiles

POST /api/vendor/openings/:id/profiles/upload

Must use Prisma transaction.

IT Vendor Frontend

Route

/vendor/openings

Table Columns

- Title
- Location
- Contract Type
- Posted Date
- Hiring Manager Name

Opening Details

/vendor/openings/:id

Must include:

- Opening details
- Upload section
- Drag-drop support
- Multiple upload

www.zelosify.com

• Soft delete support
• File preview via backend
• Light & dark theme

👨‍💼 Persona 2 – Hiring Manager

🎯 Hiring Manager User Story

I Want To:

• View my openings
• View submitted profiles
• See AI recommendation per profile
• See score + explanation
• Shortlist or reject

So That:

I can efficiently filter high-quality candidates using AI-assisted decision support.

🌓 Authentication & Authorization

Role

Role: HIRING_MANAGER

Rules

Action	Allowed
View own openings	✅
View submitted profiles	✅
Upload profiles	❌
See other manager openings	❌

www.zelosify.com

Action	Allowed
Shortlist/Reject	✅

Mandatory condition:

opening.hiringManagerId === loggedInUser.id

🤖 AI Recommendation Agent (MANDATORY LLM Tool-Using Agent, use gemini/groq for free models)

This system must implement a real LLM-based Agent with tool-calling capability.

The agent must:

1. Use an LLM that supports tool/function calling.
2. Dynamically decide when to invoke:
    - Resume Parsing Tool
    - Feature Extraction Tool
    - Skill Normalization Tool
3. Use structured output schema validation.
4. Prevent prompt injection from resume content.
5. Implement retry logic for malformed outputs.
6. Maintain internal reasoning state.
7. Log token usage and latency.
8. Persist intermediate reasoning metadata.

⚠️ Deterministic-only logic without LLM reasoning will result in rejection.

⚠️ Calling an LLM once and directly mapping output to DB without orchestration is not acceptable.

🧠 Required Agent Architecture

Controller
↓
Recommendation Service
↓

www.zelosify.com

Agent Orchestrator
↓
LLM Core (Tool-Calling Enabled)
↓
Tool Registry
- Resume Parsing Tool
- Feature Extraction Tool
- Skill Normalization Tool
- Deterministic Matching Engine
- Scoring Engine

↓
Schema Validator
↓
Decision Policy
↓
Persist Result

Rules:

- Controller must not contain business logic.
- Agent must dynamically choose tools.
- Deterministic scoring engine must be invoked as a tool.
- All tool outputs must be validated before persistence.
- The LLM must not directly read raw DB models.

Resume Parsing(Tool-Based via Agent)

Resume parsing must be implemented as a callable tool.

The LLM agent must:

- Decide when resume parsing is required.
- Retrieve file securely from S3.
- Invoke Resume Parsing Tool.
- Extract structured schema:

```
{
  experienceYears: number,
  skills: string[],
  normalizedSkills: string[],
  location: string,
  education: string[],
```

```
  keywords: string[]
}
```

Note

Resume content must never be directly injected into LLM system prompts without sanitization. Prompt injection mitigation is mandatory.

Must support:
• PDF
• PPTX

Must use backend S3 retrieval.

Feature Vector

```
{
  experienceYears: number,
  skills: string[],
  location: string,
  skillMatchScore: number,
  experienceMatchScore: number,
  locationMatchScore: number
}
```

Matching & Scoring Engine (Invoked as Tool)

The deterministic matching logic must be implemented as a separate tool callable by the LLM agent.

The agent must:

- Pass structured feature vector to matching tool.
- Receive scoring breakdown.
- Use this result in reasoning step.
- Produce explanation based on scoring + reasoning.

Hardcoded controller-level scoring without agent invocation is not acceptable

www.zelosify.com

⚙ Matching Logic (Deterministic)

Experience Logic

if candidateExp < min → 0
if within range → 1
if above max → 0.8

Skill Match

overlap / requiredSkills

Location Match

Remote → 1
Onsite mismatch → 0.5
Exact match → 1

🧮 Final Score Formula (MANDATORY)

FinalScore =
(0.5 * skillMatchScore) +
(0.3 * experienceMatchScore) +
(0.2 * locationMatchScore)

The scoring engine must:

- Be invoked as a tool by the agent.
- Return:

{
skillMatchScore,
experienceMatchScore,
locationMatchScore,
finalScore
}

The LLM must not calculate the score internally.

Deterministic scoring must remain outside the LLM to ensure explainability.

www.zelosiiy.com

Decision Thresholds

Score	Decision
≥ 0.75	Recommended
0.5–0.74	Borderline
< 0.5	Not Recommended

Agent Output Format

```
{
  "recommended": true,
  "score": 0.82,
  "confidence": 0.91,
  "reason": "Strong skill match (80%), experience within range."
}
```

Performance Constraints (STRICT)

- Max processing time per profile: 1500ms
- P95 latency < 2000ms
- No blocking API > 2 seconds
- Async execution preferred
- UI must not freeze

Latency must be stored in DB.

Recommendation Trigger

Must support:

Automatic on SUBMITTED by IT_VENDOR

zelosify.com

Hiring Manager APIs

1.Fetch Openings

GET /api/hiring-manager/openings

Only own openings.

2.Fetch Profiles

GET /api/hiring-manager/openings/:id/profiles

Must include:

- Recommendation badge
- Score
- Confidence
- Explanation
- Latency

3.Shortlist

POST /api/hiring-manager/profiles/:id/shortlist

4.Reject

POST /api/hiring-manager/profiles/:id/reject

Hiring Manager Frontend

Route

/hiring-manager/openings

zelosify.com

Opening Detail

/hiring-manager/openings/:id

Each Profile Card must show:

• File name
• Upload date
• 🟢 Recommended
• 🟡 Borderline
• 🔴 Not Recommended
• Score %
• Confidence %
• Explanation
• Processing time
• Shortlist button
• Reject button

Must support:

• Dark mode
• Responsive
• Table virtualization (>50 records)
• Skeleton loaders
• Error boundaries

📊 Observability

Must log:

• Start time
• Parsing time
• Matching time
• Final score
• Errors

Structured JSON logs only.

---

zelosify.com

ACID & Transactions

• All DB writes in Prisma transaction
• No partial recommendation write
• Idempotent re-run support

Testing Requirements

Unit Tests

• Experience boundary tests
• Skill overlap accuracy
• Score formula correctness
• Location logic
• Unauthorized access tests
• Tenant leakage tests

Integration Tests

Full flow:

Upload → Submit → Recommend → Shortlist

Performance Test

Simulate 100 profiles

Verify:
• P95 < 2000ms

Strict Failure Conditions

Assignment rejected if:

- No LLM tool-calling used
- Scoring logic directly embedded in controller
- Resume parsing done purely via regex without agent invocation
- No schema validation layer
- No prompt injection mitigation
- No retry logic on malformed LLM response
- No token usage logging

Final System Expectations

The completed system must:

• Support 2 personas
• Enforce strict RBAC
• Maintain tenant isolation
• Securely manage S3 files
• Implement deterministic AI agent
• Meet performance SLAs
• Be production-grade
• Be testable
• Be explainable

What This Assignment Evaluates

• Architecture maturity
• Backend design discipline
• AI agent reasoning
• Deterministic scoring
• Security awareness
• Multi-tenant thinking
• Performance awareness
• Clean separation of concerns

---

www.zerosify.com

This is no longer:

CRUD + Upload

This is:

Production-grade Multi-Tenant AI-Assisted Hiring Platform

Good luck.

Regards,

zelosify